Chapitre 12 : Concurrence

Sources :
Downey, The little book of semaphores
Shavit, The art of multiprocessor programming

Table des matières

N.1 Introduction 1

I Processus lourds et threads 2
I.1 Processus lourds 2
I.2 Processus légers 3

II Mon premier programme concurrent 4

III Exclusion mutuelle 5
III.1 Compteur concurrent 5
III.2 Atomicité 6
III.3 Mutex 6

IV Implémentation d'un mutex 7
IV.1 Premières idées pour un mutex à 2 fils 7
IV.2 Algorithme de Peterson 9
IV.3 Algorithme de (la boulangerie de) Lamport 10

V Sémaphores 13
V.1 Problème producteur-consommateur 13
V.2 Sémaphores 13

VI Étude de cas 15
VI.1 Interblocages 15
VI.2 Diner des philosophes 16

N.1 Introduction

Pour faire plusieurs « calcul » réellement en même temps, il faut plusieurs unités de calculs : les cœurs. Si le nombre de calculs est supérieur au nombre de cœurs, on simule l'exécution simultanée des calculs en les entrelaçant

> **Pseudo-définition**
>
> Un programme est dit **concurrent** s'il est constitué d'« unités » chacune s'exécutant séquentiellement et dont les exécutions sont simultanées ou entrelacées

I Processus lourds et threads (fils d'exécution)

I.1 Processus lourds

Définition

Un **processus (lourds)** est une instance d'un programme en cours d'exécution

Une centaine de processus en moyenne tournent sur une machine au repos

Le système d'exploitation par le biais de l'**ordonnanceur** garantit l'étanchéité entre les processus de sorte que chaque processus se comporte comme s'il était le seul à s'exécuter sur la machine.

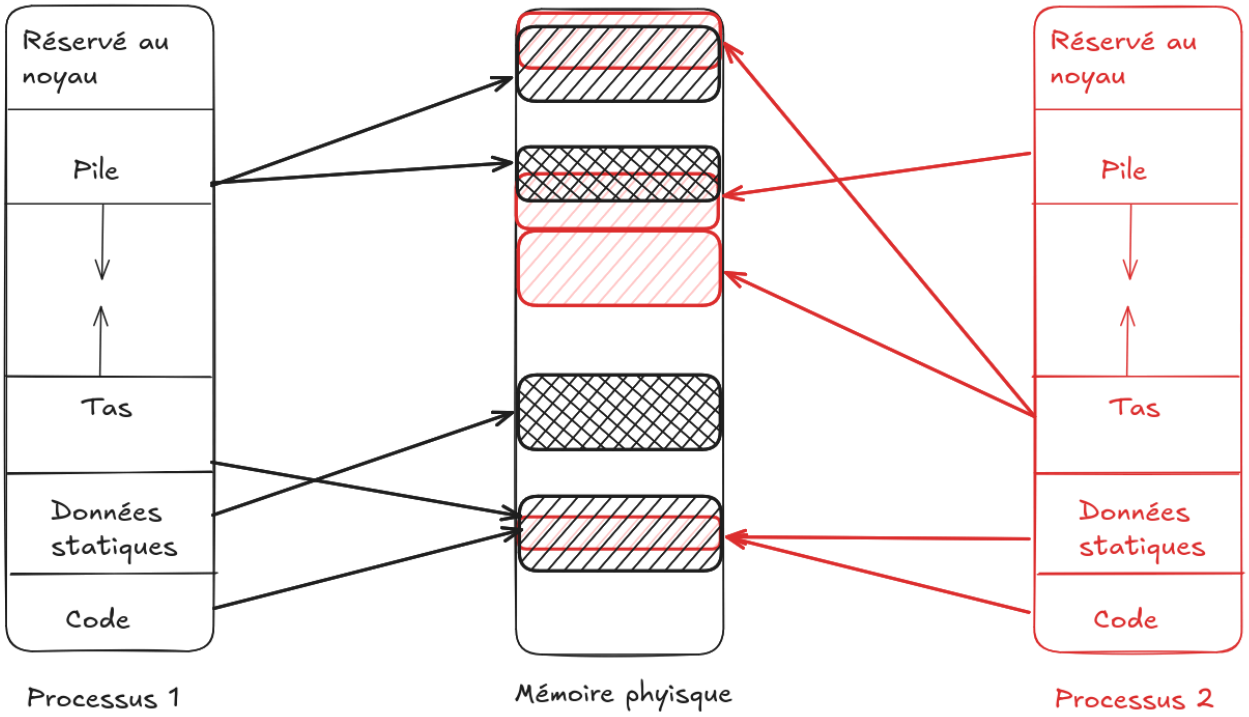

Fig. 1. – La distribution de la mémoire physique par l'ordonnanceur

Définition

La partie du système d'exploitation en charge de l'interruption, la reprise, la sauvegarde des données des processus et la correspondance mémoire physique/mémoire virtuell-t e est **l'ordonnanceur**

Définition

- L'interruption d'un processus par l'ordonnanceur pour permettre à un autre de s'exécuter est **la préemption**, on dit que le programme est **préempté**
- L'ensemble des actions de l'ordonnanceur permettant la sauvegarde des données d'un processus et la correspondance mémoire physique/mémoire vrituelle est le **changement de contexte** ou **commutation de contexte**
  Pour faire une commutation de contexte, il faut :
  - Sauvegarder l'état du program counter, des variables locales des fonctions en cours d'exécution
  - Sauvegarder la pile
  - Changer la correspondance mémoire physique/mémoire virtuelle

Deux inconvénients des processus lourds :
1. La commutation de contexte est couteuse en temps
2. Comme les processus ne paertagent rien, la seule façon de les faire communiquer est de passer par le système d'exploitation (c'est long)

I.2 Processus légers (= fils d'exécutions)

Définition

Un **fil d'exécution** est une alternative aux processus lourds qui vérifie les propriétés suivantes :
1. Ils sont « inclus » dans un processus lourd
2. Les fils d'un même processus partagent les mêmes ressources (accès au réseau, aux périphériques, aux descripteurs de fichier) et la même mémoire
   → Chaque fil a une pile et un program counter dédié (mais les autres fils peuvent a priori les consulter voire les changer)

Ainsi, on résouds certains problèmes : la commutation de contexte entre fils est rapide et ils peuvent facilement communiquer

Objectifs

- Apprendre à créer des fils d'exécution
- Étudier les principaux problèmes que posent la synchronisation des fils d'exécution et les outils pour les résoudre

II Mon premier programme concurrent

On considère qu'on a accès à deux fonctions :

- `crée_fil()` lance un fil d'exécution en lui demendant de calculer une fonction `f` sur un argument. Elle renvoie un identifiant correspondant au numéro du fil
- `joindre_fil()` prend en entrée l'identifiant d'un fil et attend que ce fil ait terminé son exécution

Avec `N` et `n` des variables globales

```
f(identifiant) :
    Afficher "le fil <identifiant> a démarré"
    Pour i = 1 à n :
        Afficher (<identifiant>, <i>)

main() :
    Afficher "Début"
    Pour i = 1 à N :
        creer_fil(f, i)
    Afficher "Pendant"
    Pour i = 1 à N :
        joindre_fil(i)
    Afficher "Fin"
```

L'ordre d'exécution des instructions de ce programme est non déterministe.
On peut visualiser tous les ordres d'exécution possible avec un graphe
(sommet = instruction, $x \to y$ si x sefait avant y)

Exemple 1: Si `N = 2` et `n = 3`

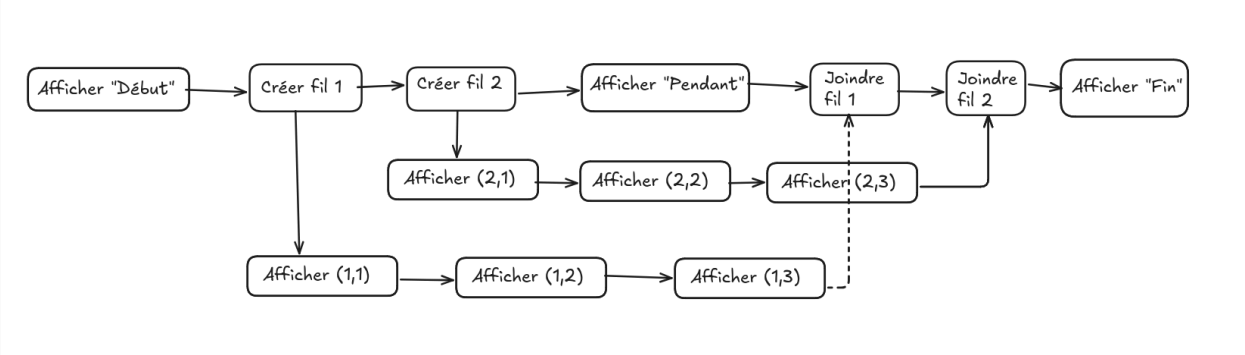

Fig. 2. – Graphe d'exécution

La différence entre un programme séquentiel et un programme concurrent est :

- Dans un programme séquentiel, les instructions sont ordonnées selon un ordre total
- Dans un programme concurrent, les instructions sont ordonnées selon un ordre partiel

(Cet ordre est celui de l'accessibilité dans le graphe : s'il y a un chemin de x vers y, x arrive avant y, si pas de chemin dans un sens ou l'autre, on ne peut pas prédir l'ordre de x et y)

Définition

Une **trace d'exécution** d'un programme est l'ordre d'exécution des instructions au cours d'une exécution de ce programme.

Un programme est concurrent s'il admet plusieurs traces d'exécutions différentes.

Exemple 2:
`Afficher "Avant"` → `Afficher 2,1` → `Afficher 2,2` → `Afficher 1,1` → `Afficher "Pendant"` → `Afficher 2,3` → `Afficher 1,2` → `Afficher 1,3` → `Afficher "Après"` est une trace d'exécution pour le programme différent.

En inversant `Afficher "Pendant"` et `Afficher 2,3` ça reste une trace d'exécution, mais pas en inversant `Afficher 2,1` et `Afficher 2,2` ni en inversant `Afficher 1,2` et `Afficher "Après"`

III Exclusion mutuelle

III.1 Compteur concurrent

Un algorithme pour compter avec une variable partagée par les fils :

```
incrémente (n) =
  for i = 1 à n
    compteur = compteur + 1

p1 = creer_fil(incrémente, 1000)
p2 = creer_fil(incrémente, 1000)
joindre(p1)
joindre(p2)
afficher compteur
```

À la fin, `compteur` ne vaut pas forcément 2000.

En effet, l'instruction `compteur = compteur + 1` se traduit pour la machine en :

```
mov compteur <registre de calcul>
add 1
mov <registre de calcul> compteur
```

Ainsi, on peut manquer des incréments, par exemple via la trace suivante :
fil 1 lit `compteur` (il vaut 0) et le stocke dans `tmp1`
fil 2 lit `compteur` (il vaut 0) et le stocke dans `tmp2`
fil 2 calcule `tmp2 ← tmp2+1 (=1)`
fil 2 écrit `compteur ← tmp2 (=1)`
fil 1 calcule `tmp1 ← tmp1+1 (=1)`
fil 1 écrit `compteur ← tmp1 (=1)`

Donc après deux incrémentations, on a augmenté le compteur que de 1.

III.2 Atomicité

> **« Définition »**
>
> Une opération est **atomique** si elle prend place ponctuellement entre le début de son appel et sa fin sans pouvoir être interrompue.
>
> Seules les instructions machines sont atomiques et on considèrera que seules les écritures ou lectures de booléens/entiers/flottants/pointeurs sont atomiques

Pour rendre des morceaux de code « atomiques », on utilise des verrous (*mutex*)

III.3 Mutex

> **Définition**
>
> Une **section critique** est une portion de code durant l'exécution de laquelle un fil doit avoir un accès exclusif à une ressource sans risque d'interruption pour garantir la correction du programme

Exemple 3: Dans le programme plus haut, `compteur ← compteur + 1` est une section critique

L'intérêt d'un mutex est de protéger les sections critiques en les rendant « atomiques ».

> **Définition**
>
> Un **mutex** (*mutual exclusion*) ou **verrou** est une primitive de synchronisation qui offre l'interace suivante :
> - `create_lock()` qui permet de créer un mutex
> - `lock(m)` qui prend (= verrouille) le mutex
> - `unlock(m)` qui le rend (= déverrouille)

Exemple 4: Pour rendre correct le compter concurrent

```
m = create_lock()
incrémente(n) :
  for i = 1 à n :
    lock(m)
    compteur++
    unlock(m)
```

Propriétés des mutex

Voici les propriétés qu'un mutex doit satisfaire :

1. **L'exclusion mutuelle** : Au plus un fil peut posséder le verrou (si plusieurs fils font un appel à `lock`, un seul peut terminer et essayer de prendre un verrou déjà possédé ne doit pas aboutir)
2. **L'absence d'interblocage** : Si au moins un fil essaie d'acquérir le verrou, au moins un y parvient en temps fini
3. **L'absence de famine** : Si un fil donné essaie d'accéder au verrou, il y arrive en temps fini

Le 1 est obligatoire pour que ça s'appelle un verrou, le 2 est obligatoire pour que ce verrou serve à quelque chose et le 3 est quand même pratique

Proposition 1: L'absence de famine implique l'absence d'interblocage et la réciproque est fausse

IV Implémentation d'un mutex

Avertissement

Cette partie est à but pédagogique, le code suivant est théoriquement correct mais en pratique ça ne marche pas. Il ne faut pas réimplanter de mutex

IV.1 Premières idées pour un mutex à 2 fils

Objectif

Implémenter les fonctions :

- `create_lock`
- `lock(m,t)` qui verrouille `m` (avec `t` l'id du fil, 0 ou 1)
- `unlock(m,t)`qui déverrouille `m` (avec `t` l'id du fil, 0 ou 1)

On considère que les fils protègent correctement leurs sections critiques (appel à `lock` puis `unlock` à chaque section critique)

Première idée : Un mutex est un enregistrement à un champ `flag` qui est un tableau à 2 cases et contient vrai en case i si et seulement si i a le verrou

```
create_lock() = {[false, false]}
unlock(m,t) =
  m.flag[t] = false
```

```
lock(m,t) =
  autre = 1-t
  while(m.flag[autre]){}
  m.flag[t] = true
```

Proposition 2: Cette implémentation ne vérifie pas l'exclusion mutuelle.

Démonstation: Voici une trace d'exécution qui le montre :

fil 0 lit `m.flag[1] (=faux)`
fil 1 lit `m.flag[0] (=faux)`
fil 1 écrit `m.flag[1] ← vrai`
fil 0 écrit `m.flag[0] ← vrai`

Deuxième idée : Pareil mais le flag idique si le fil veut le verrou

```
create_lock() = {[false, false]}
unlock(m,t) =
  m.flag[t] = false
lock(m,t) =
  autre = 1-t
  m.flag[t] = vrai
  while (m.flag[autre]){}
```

Proposition 3: Ce verrou vérifie l'exclusion mutuelle mais pas l'absence d'interblocage

Démonstation: Voici une trace d'exécution qui le montre :

fil 0 écrit `m.flag[0] ← vrai`
fil 1 écrit `m.flag[1] ← vrai`
fil 0 lit `m.flag[1] (=vrai)`
fil 1 lit `m.flag[0] (=vrai)`
Reprendre à l'étape 3

Troisième idée : On implémente un verrou comme un enregistrement à un champ `turn` qui indique le fil autorisé à prendre le verrou

```
create_lock() = {turn = 0}
unlock(m,t) =
  m.turn = 1 - t
lock(m,t) =
  autre = 1-t
  while (m.turn = autre){}
```

Proposition 4: Ce verrou vérifie l'exclusion mutuelle mais toujours pas l'absence d'interblocage

Démonstation:
Supposons par l'absurde que l'exclusion mutuelle n'est pas vérifiée.
Considérons alors la dernière exécution de `lock` pour laquelle l'exclusion n'est pas vérifiée. Le fil 0 est en section critique donc a lu `turn = 0` et le fil 1 a lu `turn = 1`. Comme le seul moment où `turn` est modifié est à la sortie de la section critique (cf. fonctionnement de `unlock`) et que les deux fils sont en section critique, $1 = 0$

Pour la présence d'interblocage, voici une trace qui le montre :
fil 1 lance un appel à `lock` (sans que fil 0 ne demande le verrou)

IV.2 Algorithme de Peterson

Un verrou est un enregistrement à deux champs :
- `want` est un tableau contenant `vrai` en case `i` si et seulement si `i` veut le verrou
- `turn` indique quel fil a la priorité

Les fils cèdent la priorité

```
create_lock() = {want = [faux, faux], turn = 0}
unlock(m,t) = m.want[t] = faux
lock(m,t) =
  autre = 1-t
  m.want[t] = vrai
  m.turn = autre
  while (m.want[autre] && m.turn = autre){}
```

Proposition 5: Un mutex implémenté via l'algorithme de Peterson vérifie l'exclusion mutuelle et l'absence de famine (donc l'absence d'interblocage)

Démonstation:

1. Supposons par l'absurde que l'exclusion mutuelle n'est pas respecté, il existe alors un moment où les deux fils sont en section critique. On peut supposer que le fil 0 est le dernier fil a avoir écrit dans `turn` avant cet instant.
   Ainsi, `turn` $= 1$, et le fil 0 est rentré dans la section critique avec `turn = 1`, donc avec `m.want[1] = faux`.
   Sauf que le fil 1 est en section critique, donc
   - soit il y est rentré avant le fil 0, donc `m.want[1]` était à `vrai` et n'as pas été remis à `faux` depuis car il n'as pas appelé `unlock`, ce qui est contradictoire.
   - soit il y est rentré après le fil 1, donc il a écrit dans `turn` après le fil 0, ce qui est aussi contradictoire

   Il y a donc une contradiction dans tous les cas, donc l'exclusion mutuelle est respectée

2. Montrons que le fil 0 ne peut pas subir de famine. Supposons qu'il est coincé dans un appel à `lock`. Alors on a `m.want[1] = vrai`, et `m.turn = 1`
   Que peut faire le fil 1 pendant ce temps?
   - S'il est au même moment coincé dans le `while`, alors `m.turn = 0`, ce qui est contradictoire. Donc le fil 1 est en train d'exécuter sa section critique
   - S'il fait un appel à `lock`, il écrit `turn = 0` et cette variable ne bougera pas jusqu'à ce que le fil 0 passe la boucle. Donc fil 0 est débloqué
   - S'il fait un appel à `unlock`, alors il met son `want`à `faux`. Même s'il le remettait à `vrai` immédiatement par un appel à `lock`, il donnerait son tour à 0. Donc fil 0 est débloqué
   - Il ne peut pas ne jamais faire d'appel à l'un des deux car il est en section critique et on suppose que les fils protègent correctement leurs sections critiques, et que leurs sections critiques terminent forcément

   Ainsi, quoi qu'il arrive, le fil 0 finira par être débloqué, donc il ne peut pas subir de famine

🔥 **Remarques pratiques sur l'algorithme de Peterson**

1. Il ne marche pas en pratique (cf. TP.20), donc on ne réimplémente pas ses propres mutex
2. Il y a une attente active dans le `lock`
3. Il n'interragit pas avec le système, l'ordonnanceur peut donc par exemple faire s'arrêter un fil qui a le verrou pour donner le cpu à un fil qui attend
4. On ne peut avoir que deux fils

IV.3 Algorithme de (la boulangerie de) Lamport

🏁 **Objectif**

Implémenter un mutex pour n fils

💡 **Idée**

Chaque fil reçoit un ticket quand il essaie d'accéder au verrou et attend tant qu'un fil qui veut le verrou a un ticket strictement plus petit que lui

‹/› Implémentation

On gère deux tableaux de taille n

- `want` tel que `want[i] = vrai` si et seulement si le fil `i` veut ou a le verrou
- `ticket` tel que `ticket[i]` contient le numéro du ticket pour le fil `i`

Pour créer et unlock, c'est facile
Pour lock :

1. Il écrit `vrai` dans `want[i]`
2. Il calcule son ticket : Il lit toute les cases de ticket dans un ordre inconnu, calcule le max m des valeurs trouvées et écrit m+1 dans ticket[i]
3. Il attend tant qu'il existe un fil $j \neq i$
   tel que `want[j] = vrai` et $(\texttt{ticket[i]},\texttt{j}) \underset{\text{lex}}{<} (\texttt{ticket[i]},\texttt{i})$

‹/› Algorithme de la boulangerie de Lamport

```
create_lock () =
  return {want = [false, ... , false], ticket = [0, ... ., 0]};

unlock (m, t) =
  m.want[t] = false;

lock(m, t) =
  m.want[t] = true;
  T = max(ticket) + 1
  m.ticket[t] = T

  while (il existe j ≠ t tq m.want[j]
         et (ticket[j],j) <_lex (ticket[t],t)){}
```

Proposition 6: Un verrou implémenté avec l'algorithme de Lamport garantit l'exclusion mutuelle

Démonstation:
Remarque préliminaire : pour tout `i`, `ticket[i]` croît au fil du temps

Supposons par l'absurde que les fils i et j soient en section critique en même temps.
On peut supposer qu'à ce moment, $(\text{ticket}[i],i) \underset{\text{lex}}{<} (\text{ticket}[j],\, j)$.
Le fil j est en section critique, donc en particulier,

1. Soit le fil i ne voulait pas y aller quand j est rentré
2. Soit le fil i avait une priorité plus grande au moment où le fil j est entré

Or,

2. Est impossible car $(\text{ticket}[i],i) \underset{\text{lex}}{<} (\text{ticket}[j],j)$ maintenant et si j a lu une ancienne valeur de ticket[i], cette valeur était encore plus petite puisque les tickets sont croissant.

1. Est impossible car i est en section critique, donc a écrit vrai dans want[i] avant que les deux fils ne soient ensemble en section critique, et cette valeur n'as pas changé car il n'as pas fait d'appel à unlock étant donné qu'il est toujours en section critique

Proposition 7: Un verrou implémenté avec l'algorithme de Lamport est sans interblocage

Démonstation: Si au moins un fil veut le verrou, comme on ne peut pas être bloqué dans le vestibule, tous les fils souhaitent le verrou se retrouvent en section d'attente au bout d'un moment. Comme $(\mathbb{N}^2, \underset{\text{lex}}{<})$ est bien fondé, il existe i tel que i veut le verrou et (ticket[i], i) est minimal parmis tous les (ticket[j], j) avec j souhaitant le verrou, et donc i obtient le verrou

Proposition 8: Si un fil F sort du vestibule avant que G n'y rentre, F accède à la section critique avant G

Démonstation: F est sorti du vestibule avant que G n'y rentre, donc F a écrit dans son ticket avant que G n'y liste. Ainsi, le ticket de G est max(ticket) + 1, avec ticket contenant celui de F, donc ticket(G) $\geq$ ticket(F) + 1 $>$ ticket(F)
Ainsi, (ticket(F),F) $\underset{\text{lex}}{<}$ (ticket(G),G)

🔥 ***Remarque***

Ceci garantit l'absence de famine car au pire, un fil attend que tous les autres inissent leur appel à lock et est ensuite nécessairement prioritaire

🔥 ***Remarque***

Le verrou implémenté par l'algorithme de Lamport a les mêmes problèmes que celui implémenté via l'algorithme de Peterson (attente active, non communication avec le système) en plus de celui de la croissance des tickets, qui demandera un reset au bout d'un moment

V Sémaphores

V.1 Problème producteur-consommateur

Définition

Dans un **problème producteur-consommateur**, deux types de fils interragissent via un buffer (tampon, souvent implémenté par une file).
Certains (les consommateurs) lisent dans le buffer et les autres (les producteurs) écrivent dedans.
Si le buffer est plein, les producteurs ne doivent pas pouvoir produire et si le buffer est vide les consommateurs ne doivent pas pouvoir consommer

Résolution du problème avec des mutex

On se place dans le cadre où le buffer st de taille infini. Pour synchroniser les fils producteurs et consommateurs :

```
on initialise un mutex pour protéger le buffer
producteurs :
  lock(m)
  produire
  unlock(m)
consommateurs :
  lock(m)
  Si le buffer n'est pas vide :
    consommer
  unlock(m)
```

V.2 Sémaphores

Définition

Un **sémaphore** est une structure de données disposant d'un compteur et de trois fonctions d'interface :

- La création qui initialise la valeur du compteur par un entier positif passé en paramètres
- incrémenter le compteur, qui peut se nommer `release`, `post`, `vendre`, `V`, `Verhogen`, `signaler`, `libérer`
- décrémenter le compteur, qui peut se nommer `acquire`, `wait`, `prendre`, `P`, `Proberen`, `réquisitionner`

Les incrémentations et décrémentations d'un sémaphores sont des opérations atomiques donc n'ont pas besoin d'être protégés

Contraintes respectées par le sémaphore

- Si un fil décrémente le compteur et qu'après cette décrémentation le compteur esst strictement négatif, le fil est mis en attente (pas active)
- Après une incrémentation, si au moins un fil est en attente, l'un est libéré et poursuit son exécution

Remarque

- Si le compteur est négatif, sa distance à 0 correspond au nombre de fils en attente
- Dans la plupart des implémentations, le fil libéré lors d'une incrémentation est celui qui attend depuis le plus longtemps (comportement premier arrivé, prémier servi)

Résolution avec des sémaphores

```
m = creer_mutex
nb_produits = creer_semaphore(0)
taille_restante = creer_semaphore(TAILLE_MAX)

producteur :
  wait(taille_restante)
  lock(m)
  produire
  unlock(m)
  post(nb_produit)

consommateurs :
  wait(nb_produit)
  lock(m)
  consommer
  unlock(m)
  post(taille_restante)
```

Proposition 9: On peut implémenter un mutex avec un sémaphore

Démonstation:

```
create_lock () =
    create_semaphore(1)

lock (m) =
    wait(m)

unlock (m) =
    post(m)
```

Avertissement

Un sémaphore et un mutex ne sont pas la même chose :

Sémaphore	**Mutex**
Comportement premier arrivé premier servi	Pas forcément
Attente réelle	Attente potentiellement active
Un fil peux signaler sans avoir pris auparavant	Un fil ne peut pas faire unlock sur un mutex qu'il n'as pas lock
Peut créer un mutex qui autorise un nombre constant (pas forcément 1) en section critique	N'autorise qu'au plus un fil en section critique

VI Étude de cas

VI.1 Interblocages

Exemple 5:

Avec `m1` et `m2` des mutex

Deux fil exécutent :
`f1 : lock(m1), lock(m2), (), unlock(m2), unlock(m1)`
`f2 : lock(m2), lock(m1), (), unlock(m1), unlock(m2)`
On a un interblocage possible possible en dépit des bonnes propriétés des verrous

Conditions de Coffman (pour détecter la présence d'interblocage)

Résultat hors-programme, permet de savoir s'il y a un interblocage mais sur sa copie il faut le prouver sans appel à cette condition par une trace d'exécution

1. Au moins une ressource en exclusion mutuelle pour plusieurs fils
2. Les fils ont une strégie wait-and-hold : si un fil a acquis une ressource il ne la relâche pas tant qu'il n'a pas toutes les ressources nécessaires pour progresser
3. Un fil ne peut relâcher une ressource que volontairement
4. Attente circulaire : il existe un cycle de fil tel que chaque fil détient une ressource nécessaire au fil suivant

Le principal levier d'action en MPI est le point 4.

VI.2 Diner des philosophes

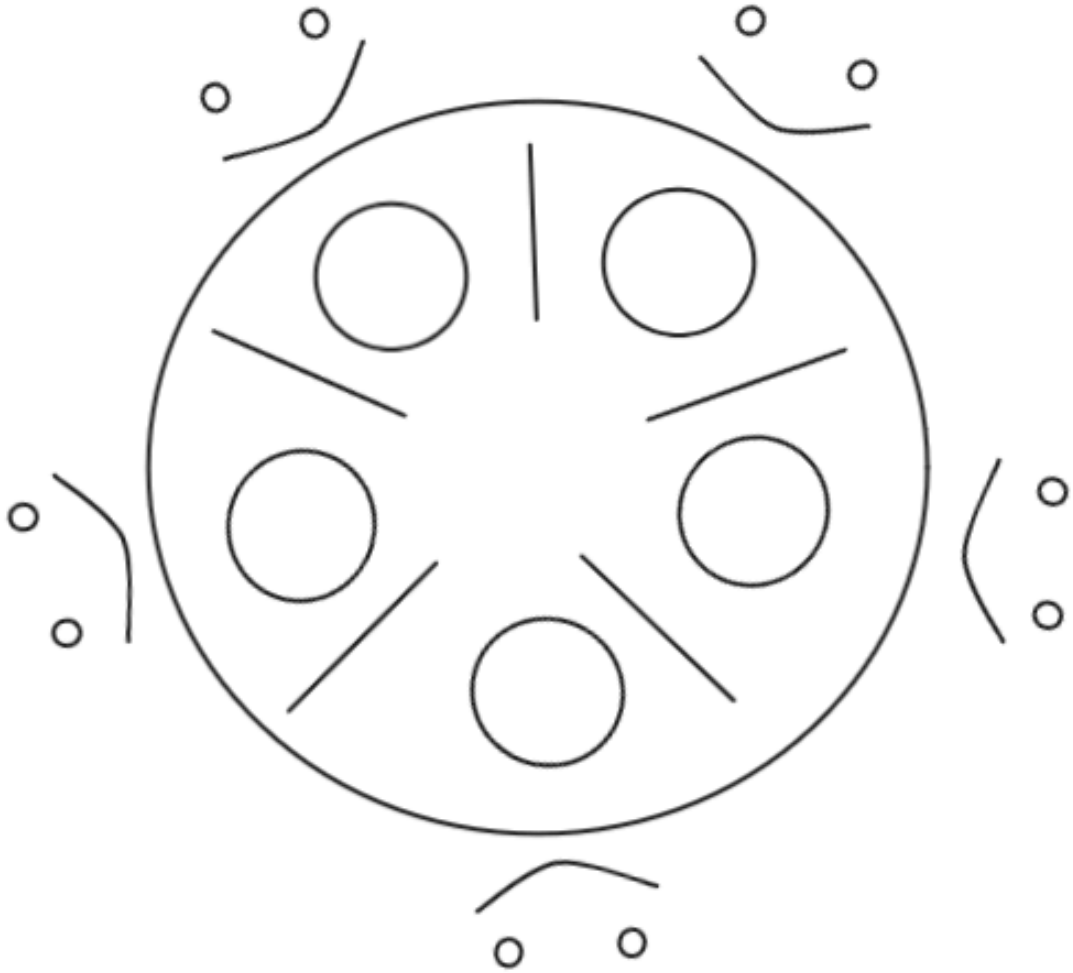

Fig. 3. – Le diner

Comment faire en sorte que :
1. Une baguette ne puisse pas être prise par deux philosophes en même temps
2. Tous les philosophes peuvent manger

Première idée : Protéger toutes les baguettes derrière un unique mutex

Problème : perte d'efficacité car au plus un philosophe peut manger alors qu'on pourrait en avoir deux

Deuxième idée : Protéger chaque baguette par un mutex

Problème : Interblocage si tout le monde prend sa baguette de gauche

Troisième idée (Dijkstra) : Introduire un sémaphore pour limiter le nombre de philosophes qui ont le droit de prendre les baguettes à 4

```
baguette = creer_sémaphore(4)
manger()
    wait(baguette)
    lock(gauche)
    lock(droite)
    manger
    unlock(gauche)
    unlock(droite)
    signaler(baguette)
```

Une idée pour trouver les interblocages avec des sémaphores

Si on a deux fils utilisant des sémaphores et qu'on veut trouver ou non l'absence d'interblocage, on peut faire un graph représentant l'ensemble des traces d'exécutions possibles. Si il existe un chemin jusqu'à la fin (le cercle) alors il est possible de finir sans interblocage. Si tous les chemins vers la fin sont possibles alors il n'y a jamais interblocage. Si aucun chemin vers la fin n'est possible alors il y a toujours interblocage.

Cet outil n'est pas au programme donc utilisable seulement pour avoir l'intuition de s'il y a interblocage ou non.

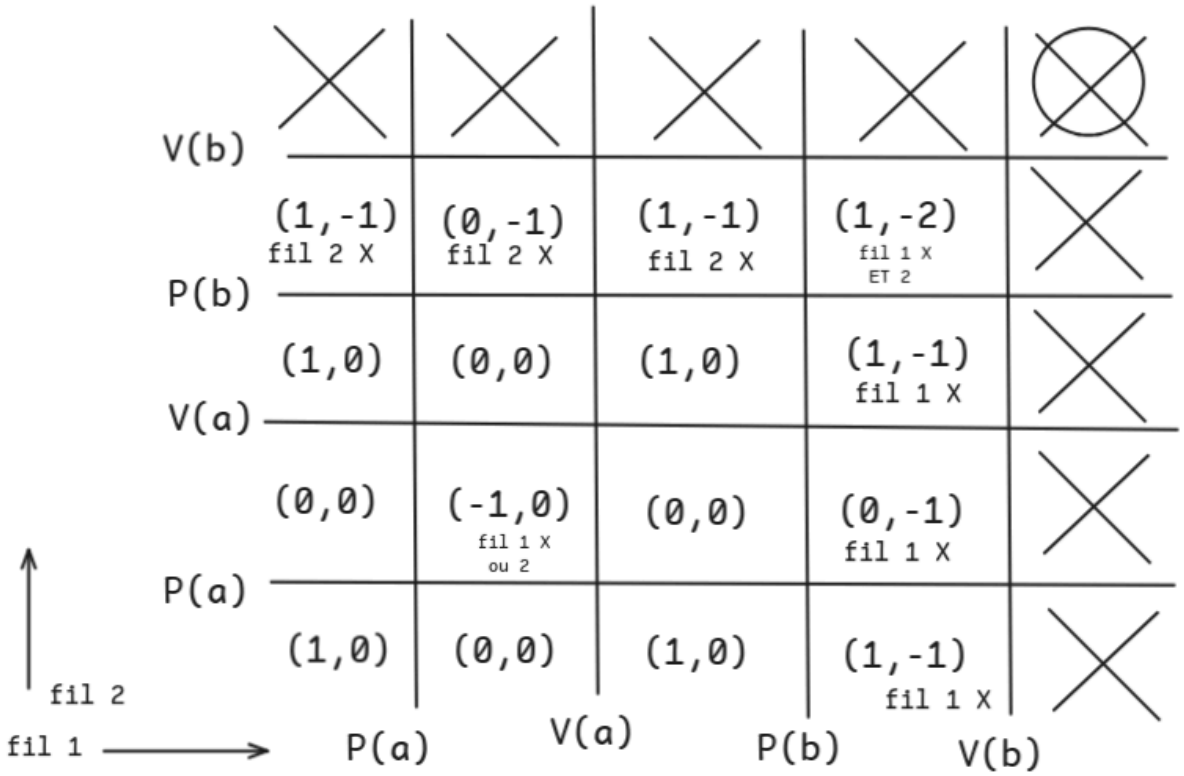

Fig. 4. – Le diagramme pour le 1. de l'ex 4 du TD